

16 장 학습 과제

과제 1. 스레드 생성과 결합

두 개의 `std::thread`로 서로 다른 범위의 정수를 출력한다. 출력 순서가 실행마다 달라질 수 있는 이유를 설명하고 모든 스레드를 `join()`한다.

`join()` 호출 하나를 제거했을 때 발생하는 결과와 `joinable()`의 상태 변화를 확인한다.

과제 2. 인수 복사와 참조 전달

정수 변수를 변경하는 함수를 스레드에서 실행한다. 값 전달, `std::ref()`, 람다의 값 캡처와 참조 캡처를 각각 적용해 원본 변수의 변경 여부를 비교한다.

참조한 지역 객체의 수명이 스레드보다 짧아지는 코드는 실행하지 않고 수명 오류가 발생하는 이유만 분석한다.

과제 3. `jthread` 협력적 중지

`std::jthread`에서 일정 간격으로 작업 번호를 출력한다. 주 스레드가 1초 뒤 `request_stop()`을 호출하고 작업 스레드가 요청을 확인해 종료하도록 구성한다.

중지 요청 확인을 제거했을 때 자동 결합이 프로그램 종료를 지연시키는 이유를 설명한다.

과제 4. 데이터 경쟁과 뮤텝스

두 스레드가 하나의 카운터를 각각 100만 번 증가시키는 프로그램을 작성한다. 동기화가 없는 버전, `std::mutex`와 `std::lock_guard`를 사용하는 버전, `std::atomic<int>`를 사용하는 버전을 비교한다.

실행 시간과 최종값을 여러 번 기록하되 데이터 경쟁 버전의 결과를 정상 동작으로 해석해서는 안 되는 이유를 작성한다.

과제 5. 일관된 상태 반환

잔액과 거래 횟수를 멤버로 가진 클래스를 작성한다. 입금 함수는 두 값을 함께 변경하고 조회 함수는 두 값을 한 번의 잠금 안에서 복사해 반환한다.

두 값을 각각 조회하는 함수로 분리했을 때 호출자가 서로 다른 시점의 값을 받을 수 있는 과정을 설명한다.

과제 6. 두 객체 사이의 전송

두 계좌 객체 사이에서 금액을 이동하는 함수를 구현한다. `std::scoped_lock`으로 두 뮤텝스를 함께 잠그고 같은 객체가 두 인수로 전달된 경우도 처리한다.

두 뮤텝스를 반대 순서로 직접 잠그는 버전에서 교착 상태의 네 조건이 어떻게 성립하는지 분석한다.

과제 7. 스레드 안전한 큐

`Push()`, `TryPop()`, `WaitPop()`, `Stop()`을 가진 작업 큐를 작성한다. `WaitPop()`은 조건 변수로 작업 또는 종료 상태를 기다린다.

종료 시 남은 작업을 모두 처리하는 정책과 즉시 폐기하는 정책을 각각 구현하고 반환 규칙의 차이를 문서화한다.

과제 8. 시간 제한 대기

작업 큐에 `WaitPopFor()`를 추가한다. 제한 시간 안에 작업이 들어오면 값을 반환하고 시간 초과나 종료 상태이면 구분 가능한 결과를 반환한다.

`std::optional<Task>`만으로 시간 초과와 종료를 구분할 수 있는지 검토하고 필요하면 `std::variant`나 열거형 결과 타입을 설계한다.

과제 9. **promise**와 **future**

작업 스레드가 파일 크기를 계산해 `std::promise<std::uintmax_t>`로 전달한다. 파일 접근 실패는 `set_exception()`으로 전달하고 호출부에서 `future::get()`의 예외를 처리한다. 값이나 예외를 설정하지 않고 `promise`가 소멸했을 때의 오류도 확인한다.

과제 10. **packaged_task** 작업 큐

서로 다른 정수 계산을 `std::packaged_task<int>`로 만들고 큐에 저장한다. 작업 스레드가 큐에서 하나씩 꺼내 실행하고 호출자는 각 `future`에서 결과를 받는다. `packaged_task`가 스레드를 직접 생성하지 않는다는 점을 코드의 실행 주체와 연결해 설명한다.

과제 11. **async** 실행 정책

같은 계산 함수를 `std::launch::async`와 `std::launch::deferred`로 실행한다. 함수 내부와 호출부의 스레드 ID를 출력해 실행 위치를 비교한다. 정책을 생략한 호출은 특정 실행 방식을 가정하지 않고 관찰 결과만 기록한다.

과제 12. 원자적 상태 전환

`Idle`, `Running`, `Completed` 상태를 정수 또는 열거형 기반 `std::atomic`으로 표현한다. `compare_exchange_strong()`을 이용해 `Idle`에서 `Running`으로 한 스레드만 전환하도록 구현한다. 실패할 때 기대값 인수가 실제 상태로 변경되는 과정을 출력한다.

과제 13. **atomic wait**와 **notify**

소비자 스레드가 `std::atomic<int>` 값이 0에서 바뀔 때까지 `wait()`하고 생산자가 값을 기록한 뒤 `notify_one()`을 호출하도록 구성한다. 같은 문제를 조건 변수로 구현하고 상태 표현, 여러 조건 확장과 코드 복잡도를 비교한다.

과제 14. **latch** 시작선

작업 스레드 네 개가 각자 준비 작업을 마친 뒤 동시에 본 작업을 시작하도록 `std::latch`를 사용한다. `count_down()`과 `wait()`를 분리한 형태와 `arrive_and_wait()`를 사용한 형태를 비교한다.

과제 15. **barrier** 단계 처리

여러 스레드가 계산, 결과 교환, 후처리의 세 단계를 반복하도록 `std::barrier`를 사용한다. 어느 스레드도 다른 스레드가 같은 단계를 끝내기 전에 이후 단계로 진행하지 않게 한다.

단계 완료 함수에서 현재 단계 번호를 증가시키도록 구성한다.

과제 16. **semaphore** 자원 제한

동시에 두 작업만 사용할 수 있는 가상 네트워크 연결 풀을 `std::counting_semaphore<2>`로 표현한다. 다섯 스레드가 연결을 획득하고 일정 시간 사용한 뒤 반환한다.

뮤텍스 하나로 전체 작업을 직렬화한 경우와 동시에 두 작업을 허용하는 세마포어의 차이를 설명한다.

과제 17. 병렬 알고리즘의 부작용

큰 벡터의 각 원소를 제공하는 `std::transform`을 순차 정책과 병렬 정책으로 실행해 시간을 측정한다.

람다에서 공유 합계를 직접 변경하는 코드가 안전하지 않은 이유를 작성하고 `std::reduce()`를 이용한 합계로 교체한다.

과제 18. 스레드 안전한 인터페이스

`Empty()`, `Front()`, `Pop()`을 각각 잠그는 큐를 작성한 뒤 세 호출을 조합해도 안전하지 않은 실행 순서를 작성한다.

검사와 제거를 하나의 `TryPop()`으로 합치고 값 또는 빈 상태를 반환하도록 수정한다.

과제 19. 종료 가능한 작업자 클래스

내부에 작업 큐, 조건 변수와 `std::jthread`를 가진 `Worker` 클래스를 작성한다. 생성자는 작업 스레드를 시작하고 소멸 과정에서는 새 작업 접수를 막고 대기자를 깨운 뒤 스레드가 끝나도록 한다.

멤버 선언 순서가 소멸 순서와 스레드 안전성에 미치는 영향을 설명한다.

과제 20. 동시성 설계 검토

파일 로딩, 이미지 디코딩, 화면 객체 생성으로 이루어진 로딩 시스템을 설계한다. 각 단계에서 실행할 스레드, 데이터 소유권, 결과 전달 방식과 취소 정책을 정한다.

공유 상태, 잠금 대상, 조건 변수의 조건식, 프로그램 종료 순서를 **UML** 다이어그램으로 표현한다.